DESIGN AND ANALYSIS OF ALGORITHMS

Campus Navigation and Path Finding System

DAA 5th Semester Mini Project Report

Bachelor of Technology
Information Technology

Name : Parth Thumar | 12402080601102

Rahul Vanani | 12402080601112

Mann Patel | 12502080603008

Acknowledgement

We acknowledge the guidance provided through the Design and Analysis of Algorithms mini-project requirements and the academic learning that supported graph modeling, algorithm implementation, testing and evaluation. We also acknowledge the Python standard library, Matplotlib and NumPy documentation used by the supplied implementation. The system uses a fixed coordinate-based campus graph and locally generated benchmark graphs so that algorithmic behavior can be inspected and reproduced without external services.

Abstract

Campus navigation can be formulated as a shortest-path problem on a weighted graph. This project presents an academic prototype in which named campus locations are vertices, two-way walkways are edges, and edge weights are Euclidean lengths calculated from stored coordinates. The supplied single-file Python implementation compares BFS, DFS, Dijkstra's Algorithm and A* Search. Dijkstra and A* are the principal weighted shortest-path algorithms, while BFS and DFS serve as educational baselines.

The implementation uses adjacency lists, predecessor maps, explored lists, visited sets and binary min-heaps. A* evaluates f(n)=g(n)+h(n), where g(n) is the distance travelled and h(n) is straight-line Euclidean distance to the destination. The program measures route cost, explored nodes and execution time for a user-selected query, then benchmarks all four algorithms on deterministic connected graphs of 10, 50, 100 and 500 nodes, using 200 repetitions per algorithm and graph size. For the default Main Gate to Hostel B query, Dijkstra and A* both produced a route cost of 591.4 m; A* explored 5 nodes compared with Dijkstra's 12. In the generated benchmark A* matched Dijkstra's cost at all sizes and explored fewer nodes, while measured time varied with graph size. The project is deliberately an offline academic simulation and does not claim live GPS, traffic, GIS integration, mobile positioning or production navigation capability.

Table of Contents

1. Introduction
2. Problem Definition
3. Motivation and Academic Application
4. Objectives and Scope
5. Background and Existing Approach
6. System Requirements
7. Proposed System and Architecture
8. Overall System Flow
9. Graph Representation
10. Data Structures
11. Dijkstra’s Algorithm
12. A* Search
13. Dijkstra versus A* Comparison
14. Implementation Details
15. User Interface and Working Model
16. Testing
17. Experimental Setup and Methodology
18. Results and Performance Comparison
19. Results Interpretation
20. Complexity Analysis
21. Theoretical versus Practical Comparison
22. Advantages and Limitations
23. Future Scope
24. Conclusion
25. References

Page numbers correspond to this generated document. If the report is edited substantially, refresh the contents list before submission.

List of Figures

Figure 1. System architecture of the implemented prototype
Figure 2. Overall system flow from input to performance analysis
Figure 3. Illustrative weighted graph and shortest-path model
Figure 4. Campus network and generated route visualization
Figure 5. Route comparison output for the four algorithms
Figure 6. Average execution time versus graph size
Figure 7. Average nodes explored versus graph size

List of Tables

Table 1. Project objectives
Table 2. Software requirements
Table 3. Data structures used
Table 4. Dijkstra and A* comparison
Table 5. Verification and test status
Table 6. Experimental methodology
Table 7. Benchmark summary
Table 8. Complexity analysis

1. Introduction

Campus environments contain buildings, intersections, pathways and alternative routes. A visitor may know a destination but not the most efficient sequence of connected locations. From a Design and Analysis of Algorithms viewpoint, the problem is an instance of shortest-path computation on a weighted graph.

The supplied program represents locations with coordinates and roads with bidirectional connections. The distance between two connected locations becomes the edge weight. Dijkstra expands the currently cheapest unsettled node, while A* adds a geometric estimate of remaining distance. BFS and DFS demonstrate why fewest edges or an early-found path need not minimize weighted distance.

The objective is to provide a transparent, offline learning system that combines graph modeling, algorithm design, path reconstruction, complexity analysis, benchmarking and visualization. It is not presented as a live campus navigation platform.

2. Problem Definition

2.1 Problem statement

Given a source and destination among the fixed campus locations, compute a route through the weighted campus graph and report its total length. The same query is evaluated by four algorithms so route quality and search effort can be compared.

2.2 Input

- Source and destination location names entered at the terminal.
- The fixed LOCATIONS coordinate dictionary and ROADS list.
- Rounded Euclidean edge lengths produced by build_graph().
- For benchmarking, connected generated graphs of 10, 50, 100 and 500 nodes.

2.3 Output

- Ordered path from source to destination.
- Total path cost in metres.
- Explored-node order and count.
- Measured execution time and Matplotlib visualizations.

2.4 Constraints and assumptions

- Static graph and offline operation.
- Two-way roads and non-negative edge weights.
- Exact location-name matching.
- Synthetic benchmark coordinates; no real geographic claim.

2.5 Objective function

The route objective is to minimize the sum of edge weights. The experimental objective is to compare algorithms on the same graph, source and destination using cost, explored nodes and time.

3. Motivation and Academic Application

A campus graph provides a compact setting for studying a problem that scales from a few locations to hundreds of generated nodes. The abstraction separates the route algorithm from the display code and makes the effect of queues, stacks, heaps and heuristics observable. The project is academically relevant because it turns textbook shortest-path reasoning into a runnable experiment.

A real campus service could eventually incorporate validated map data or accessibility constraints, but those features are outside the supplied implementation. The current system should be interpreted only as an academic simulation.

4. Objectives and Scope

No.	Objective
1	Represent campus locations and walkways as a weighted adjacency-list graph.
2	Implement BFS, DFS, Dijkstra and A* from scratch.
3	Reconstruct and display source-to-destination paths.
4	Compare path cost, explored nodes and execution time.
5	Benchmark deterministic generated graphs.
6	Relate practical results to DAA complexity and correctness.

The scope includes graph construction, four search routines, terminal input, benchmark generation and live Matplotlib windows. It excludes live GPS, GIS, traffic, accounts, databases, mobile deployment and operational navigation.

5. Background and Existing Approach

Graphs consist of vertices and edges. An adjacency list stores only existing connections and uses O(V+E) storage. In this project roads are undirected because build_graph() inserts both directions. BFS is suitable for equal edge costs, DFS is a traversal baseline, Dijkstra is the weighted baseline, and A* is a heuristic-guided weighted search.

Manual route selection does not provide a systematic optimality guarantee. The implemented comparison therefore focuses on algorithmic behavior instead of replacing a specific commercial product.

6. System Requirements

Category	Requirement
Hardware	Any machine capable of running Python and opening Matplotlib windows; exact hardware not recorded.
Software	Python with matplotlib and numpy installed.
Input	Exact campus location names or Enter for defaults.
Processing	Dictionary adjacency list, deque/list frontiers, heapq priority queues.
Output	Terminal metrics, live maps, route comparison and performance graphs.
Limitations	No network service, database, GPS, GIS or traffic feed.

7. Proposed System and Architecture

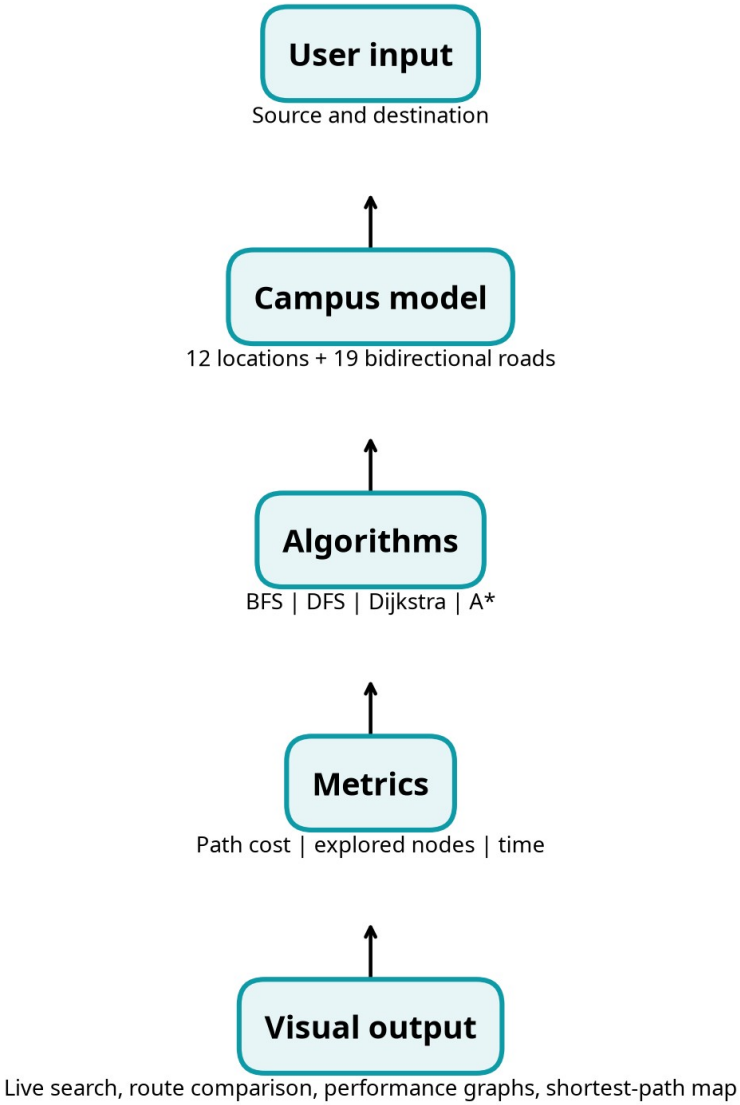


Figure 1. System architecture of the implemented prototype

This architecture follows the supplied functions: terminal input is checked against LOCATIONS, build_graph() creates the weighted adjacency list, the four algorithms produce metrics, and visualization functions display the results.

8. Overall System Flow

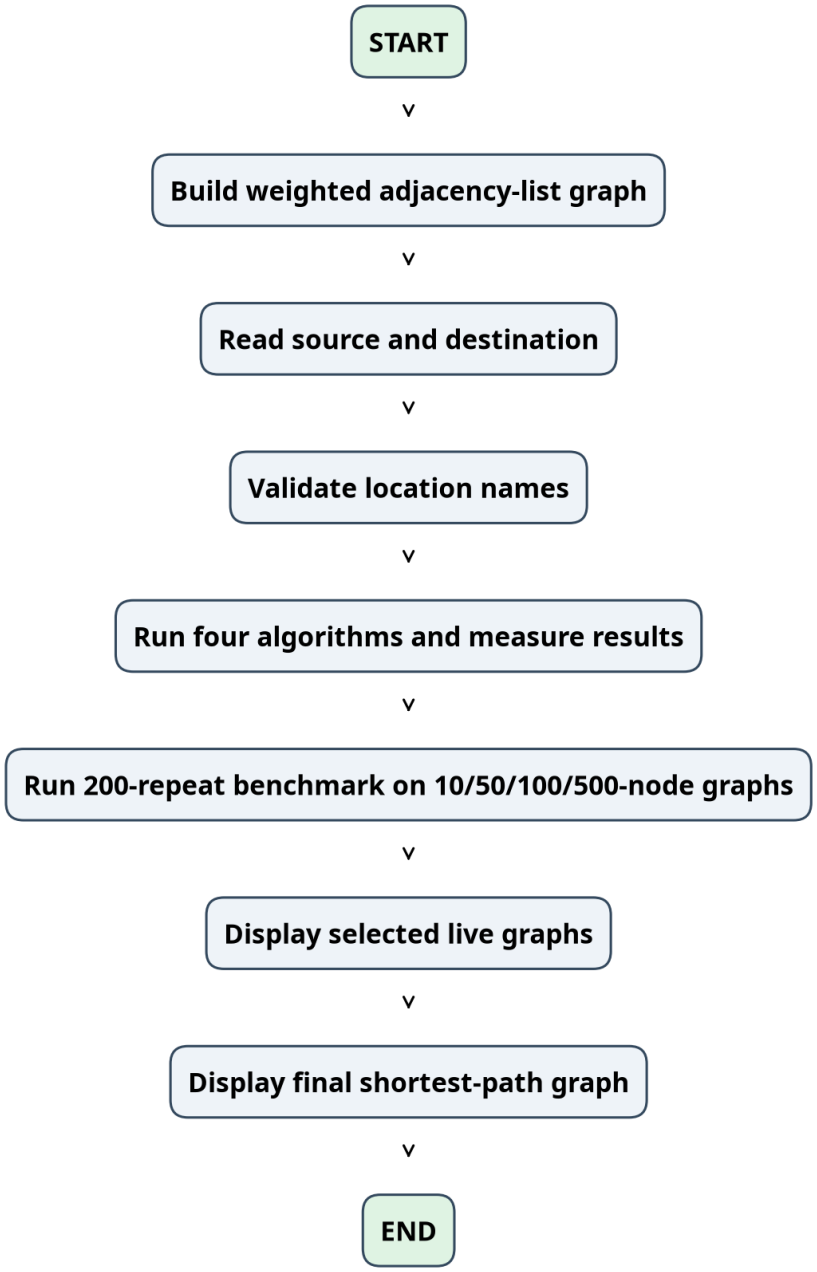


Figure 2. Overall system flow from input to performance analysis

The flow is derived from main(): construct the campus graph, read and validate the query, run all algorithms, run the benchmark, show selected plots and finish with the shortest-path graph.

9. Graph Representation

LOCATIONS contains 12 named vertices with (x,y) coordinates. ROADS contains 19 endpoint pairs. build_graph() calculates each road's Euclidean length, rounds it to one decimal place, and stores both directions. The graph consequently has 38 adjacency entries. random_graph() creates connected benchmark graphs by linking each new node to its nearest earlier node and adding edges to up to three nearest neighbors.

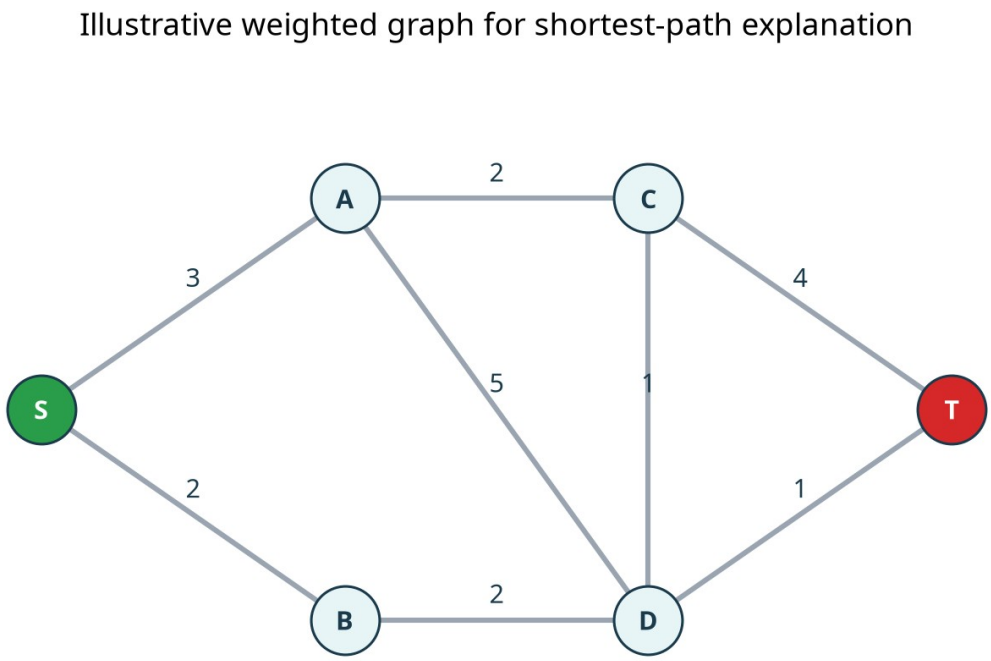


Figure 3. Illustrative weighted graph and shortest-path model

This diagram is an explanatory graph and is not a claim about the exact campus topology. It shows source, destination, alternative routes and edge costs.

10. Data Structures

Structure	Purpose	Used by
Adjacency-list dictionary	Stores neighbors and edge weights	All algorithms
Parent dictionary	Stores predecessor for path reconstruction	All path searches
deque	FIFO frontier	BFS
List stack	LIFO frontier	DFS
heapq min-heap	Selects smallest priority	Dijkstra and A*
done/visited set	Prevents repeated expansion	DFS, Dijkstra, A*
Explored list	Records search order and count	All algorithms

11. Dijkstra's Algorithm

Dijkstra initializes best distances to infinity, sets the source to zero, and pushes the source into a min-heap. It repeatedly extracts the smallest tentative distance, skips already completed nodes, relaxes every neighbor and updates parent pointers when a shorter route is found. The target is returned when extracted.

```
DIJKSTRA(G,s,t)
  best[v] ← ∞ for every v; best[s] ← 0
  parent[s] ← NIL; heap ← {(0,s)}; done ← ∅
  while heap ≠ ∅:
    (d,u) ← extract-min(heap)
```

```
  if u ∈ done: continue
  add u to done and explored
  if u = t: return rebuild_path(parent,t)
  for (v,w) in G[u]:
    if d+w < best[v]: update best[v], parent[v], heap
return no path
```

With non-negative weights, the smallest unsettled distance is final. With an adjacency list and binary heap, the worst-case order is O((V+E) log V), with O(V+E) graph storage and O(V+E) possible auxiliary state.

12. A* Search

A* uses f(n)=g(n)+h(n). In this implementation g(n) is the accumulated edge length and h(n) is distance(n,destination,pos), the Euclidean distance to the target. The heap stores f values and parent updates follow the same relaxation pattern as Dijkstra.

```
A_STAR(G,s,t,pos)
  g[v] ← ∞ for every v; g[s] ← 0
  heap ← {(h(s),s)}; parent[s] ← NIL; done ← ∅
  while heap ≠ ∅:
    (_,u) ← extract-min(heap)
    if u ∈ done: continue
    add u to done and explored
    if u = t: return rebuild_path(parent,t)
    for (v,w) in G[u]:
      if g[u]+w < g[v]: update g[v], parent[v], heap with g[v]+h(v)
  return no path
```

Because road weights are straight-line distances between connected coordinates, the straight-line target distance is a lower bound for the intended geometric model. The guarantee is conditional on this weight interpretation; it should not be assumed for arbitrary future weights. The worst-case heap-based order remains O((V+E) log V).

13. Dijkstra versus A* Comparison

Parameter	Dijkstra	A*
Priority	g(n)	g(n)+h(n)
Heuristic	None	Euclidean distance to destination
Optimality	Non-negative weights	Under heuristic/weight assumptions
Direction	Expands by source distance	Directed toward goal
Data structure	Adjacency list + heap	Adjacency list + heap
Project result	Optimal baseline	Same cost, fewer nodes in evidence

A* can explore fewer nodes because the heuristic prioritizes geometrically promising vertices. It is not necessarily faster in every run: heuristic calculations, heap operations, Python overhead and small graph sizes influence measured time.

14. Implementation Details

The project is a single Python file: campus_navigation_live(2).py. It imports heapq, math, random, time, deque, Matplotlib and NumPy. The functions are organized into campus data, graph utilities, four algorithms, measurement, visualizations and main(). The program does not expose a backend or web frontend.

Uploaded project artifact
- *campus_navigation_live(2).py*
  - *LOCATIONS and ROADS*
  - *build_graph and random_graph*
  - *bfs, dfs, dijkstra, astar*
  - *run_all and benchmark*
  - *draw_map and visualization functions*
  - *main entry point*

15. User Interface and Working Model

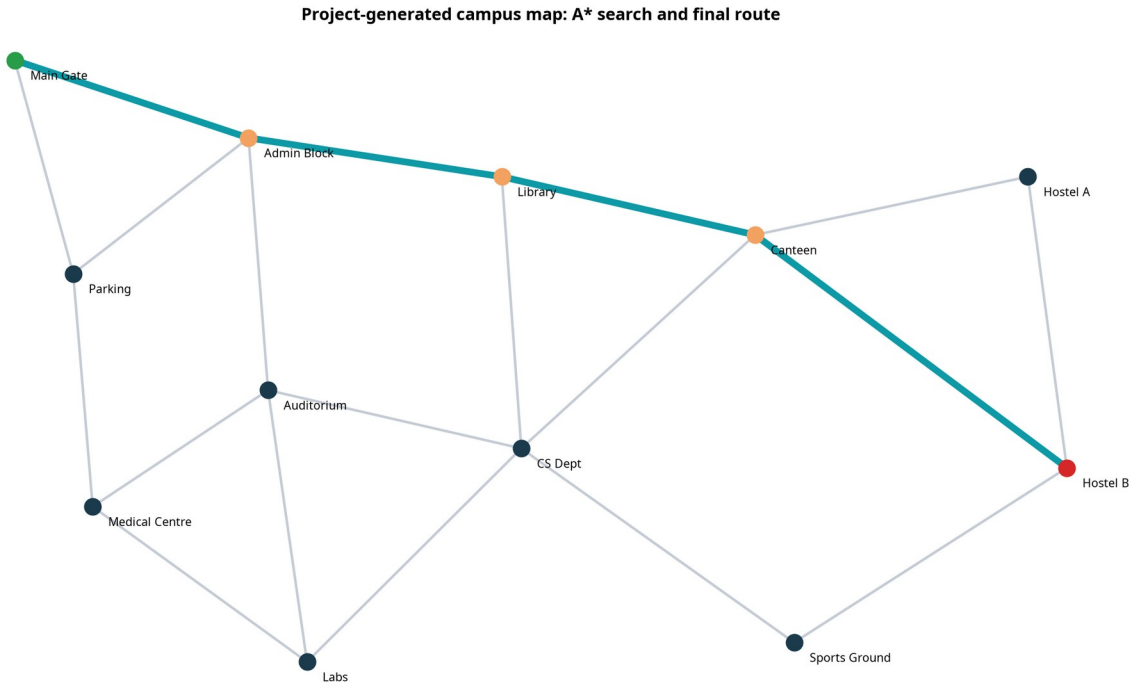


Figure 4. Campus network and generated route visualization

This project-generated figure calls the supplied draw_map() with the default Main Gate to Hostel B query. Green is the source, red is the destination, orange marks explored locations and teal shows the route.

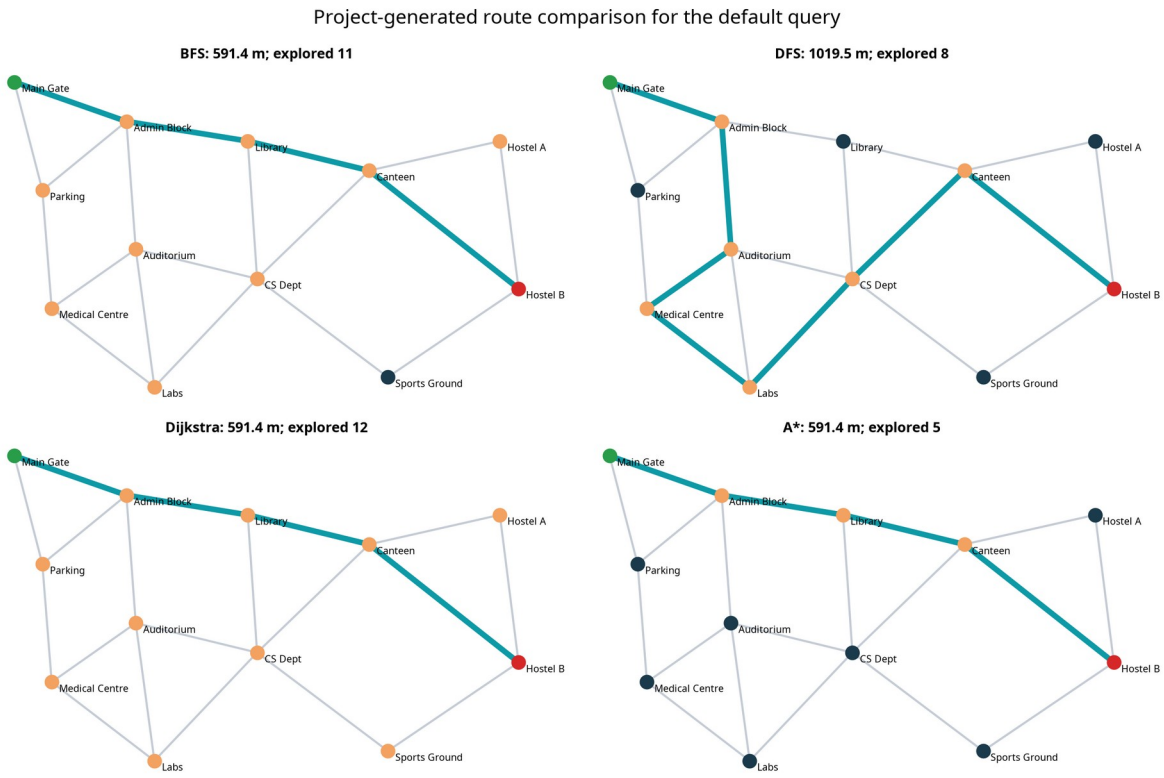


Figure 5. Route comparison output for the four algorithms

The four panels use the result dictionaries returned by run_all(). This is a generated visualization because no original screenshot file was supplied with the upload.

The interface is terminal-driven. The user enters exact source and destination names, observes printed metrics, chooses one or more graph windows, and the program finishes with the shortest-path graph.

16. Testing

No separate test files or pytest configuration were supplied. The following table records evidence-supported verification without claiming a pre-existing automated test suite.

ID	Check	Status
V-01	Fixed graph builds with 12 locations and 19 roads	Verified during evidence generation
V-02	Default Main Gate to Hostel B query executes	Verified during evidence generation
V-03	Dijkstra and A* costs match on default query	Verified: 591.4 m each
V-04	Benchmark executes four graph sizes and 200 repeats	Verified
V-05	Invalid-name branch exists in main()	Code inspected; not separately executed
V-06	Unreachable destination behavior	Not tested in supplied artifact
V-07	Source equals destination behavior	Not tested in supplied artifact

Future automated tests should cover unreachable graphs, source=destination, invalid names, path correctness and repeated benchmark consistency.

17. Experimental Setup and Methodology

Item	Recorded method
Graph sizes	10, 50, 100, 500 nodes
Graph generator	Connected coordinate graph with nearest-neighbor additions
Seed	1
Repetitions	200 per algorithm and size
Source/destination	0 and n−1 for generated graphs
Timing	time.perf_counter(); elapsed time divided by 200
Metrics	Average time, explored nodes and path cost
Hardware/software versions	Not explicitly recorded in project

The evidence was generated by importing the uploaded implementation without changing its algorithms. Timing is machine-dependent and represents the execution environment used for this report.

18. Results and Performance Comparison

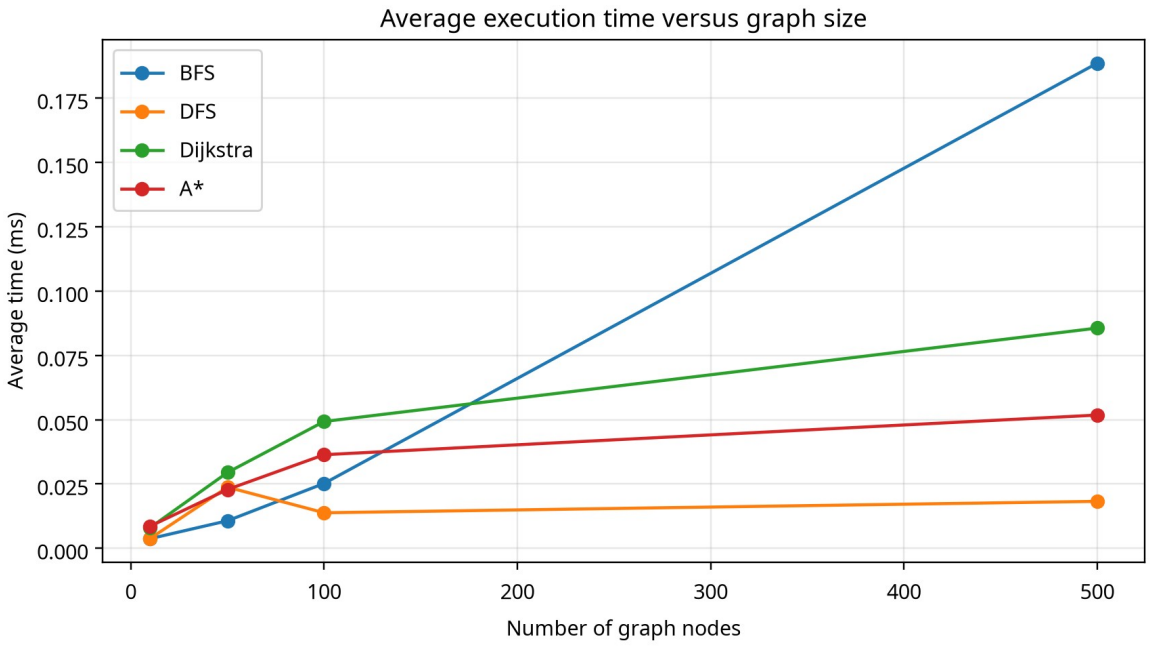


Figure 6. Average execution time versus graph size

The graph uses the benchmark() output for four algorithms, four graph sizes and 200 repetitions per size.

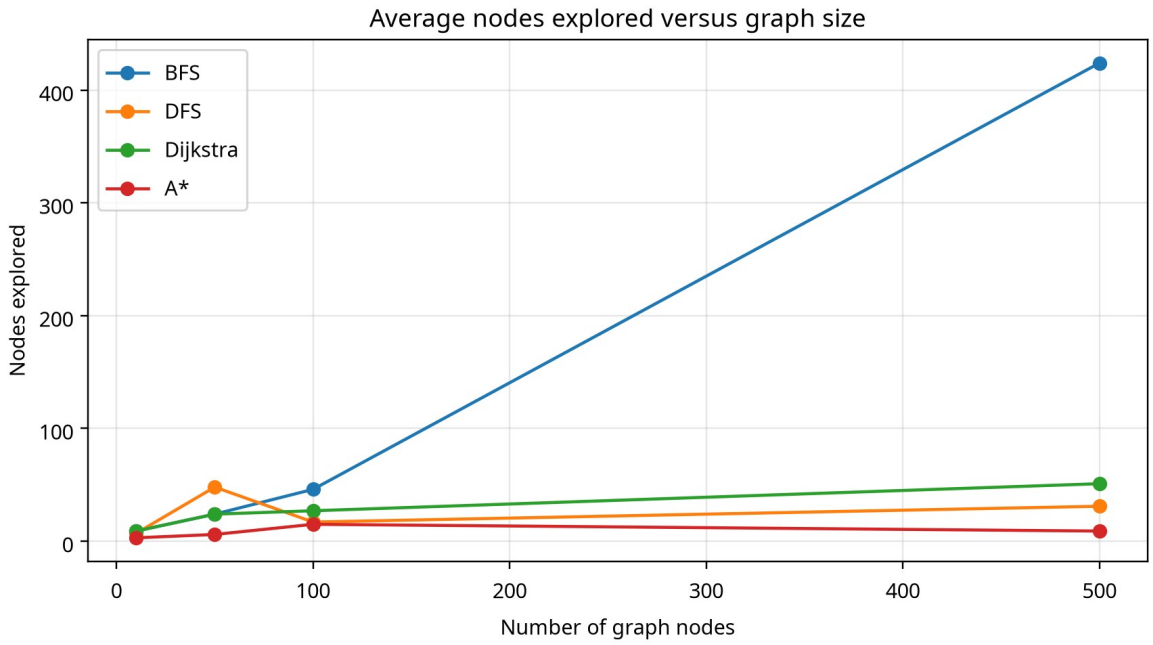


Figure 7. Average nodes explored versus graph size

The graph uses the explored list length returned by each algorithm in the benchmark routine.

Nodes	Algorithm	Avg time (ms)	Explored	Cost (m)
10	BFS	0.0038	9	1127.6
10	DFS	0.0038	7	2307.7
10	Dijkstra	0.0080	9	1127.6
10	A*	0.0086	3	1127.6
50	BFS	0.0107	24	1265.6
50	DFS	0.0238	48	1320.3
50	Dijkstra	0.0295	24	740.2
50	A*	0.0229	6	740.2
100	BFS	0.0252	46	661.3
100	DFS	0.0138	17	1325.7
100	Dijkstra	0.0493	27	617.3
100	A*	0.0364	15	617.3
500	BFS	0.1886	424	421.1
500	DFS	0.0183	31	1296.7
500	Dijkstra	0.0856	51	251.5
500	A*	0.0518	9	251.5

19. Results Interpretation

A* explored fewer nodes than Dijkstra at all tested sizes: 3 versus 9, 6 versus 24, 15 versus 27, and 9 versus 51. A* and Dijkstra matched path cost at every size. At 10 nodes A* was slightly slower than Dijkstra in the recorded run, while at 50, 100 and 500 nodes it was faster. This mixed timing result is more informative than claiming that A* is always faster.

BFS matched the optimal cost only at the 10-node benchmark. DFS produced longer costs at every benchmark size. This confirms that BFS and DFS are useful baselines but are not weighted shortest-path algorithms. The

results depend on graph structure, adjacency ordering, runtime overhead and the single seeded graph chosen for each size.

20. Complexity Analysis

Algorithm	Best/typical	Worst case	Auxiliary space	Graph storage
BFS	Immediate target O(1); traversal O(V+E)	O(V+E)	O(V)	O(V+E)
DFS	Immediate target O(1); traversal O(V+E)	O(V+E)	O(V)	O(V+E)
Dijkstra + heap	Early target may stop sooner	O((V+E) log V)	O(V+E)	O(V+E)
A* + heap	Depends on heuristic direction	O((V+E) log V)	O(V+E)	O(V+E)

V is the number of vertices and E is the number of represented adjacency entries. The implementation may hold multiple heap entries for improved tentative values; done prevents repeated expansion.

21. Theoretical versus Practical Comparison

Asymptotic complexity abstracts constants and machine behavior. The measured function includes dictionary lookups, tuple handling, lambda calls, heap operations, list appends, heuristic calculations and path-cost evaluation. Small graphs are particularly sensitive to timer precision and interpreter overhead. A lower explored-node count is therefore not a guarantee of lower wall-clock time.

- Graph density changes neighbor relaxations.
- Heuristic quality changes A* search direction.
- Python runtime and background processes affect timing.
- One seeded graph per size does not represent every possible graph.
- The benchmark excludes interactive plotting from the timed loop.

22. Advantages and Limitations

Advantages	Limitations
Clear graph abstraction	Static embedded campus map
Four algorithms in one program	Synthetic and small real-world model
Readable path and metrics	No GPS, GIS, traffic or closures
Reproducible seeded benchmark	Hardware/version metadata not recorded
Live educational visualization	Plots are not automatically exported by the app

The system is an academic prototype, not a production navigation service. Its edge lengths are meaningful only within the coordinate model in the Python file.

23. Future Scope

- Replace the fixed graph with validated campus GIS data.
- Add automated unit and integration tests.
- Export benchmark CSV/JSON and figures automatically.
- Add accessibility-aware routing and multiple route alternatives.
- Support temporary closures or dynamic weights.
- Record full environment metadata with benchmark runs.
- Consider mobile or browser deployment only as future engineering work.

24. Conclusion

The Campus Navigation and Path Finding System demonstrates the complete DAA workflow for a weighted graph: model locations and roads, choose data structures, implement search, reconstruct paths, measure performance and interpret results. The supplied Python program makes the comparison concrete through BFS, DFS, Dijkstra and A* as well as live Matplotlib views.

For the default query, Dijkstra and A* both returned a 591.4 m route, with A* exploring 5 nodes versus Dijkstra's 12. Across the generated benchmark A* retained the same path costs as Dijkstra and explored fewer nodes, but timing depended on graph size and runtime overhead. The result supports a balanced conclusion: A* can reduce search effort when its heuristic is useful, but neither algorithm should be declared universally faster.

The work remains limited to a static academic simulation. Even so, it provides an appropriate fifth-semester Information Technology project for studying shortest-path correctness, adjacency lists, priority queues, heuristic search, asymptotic complexity and experimental evaluation.

25. References

1. E. W. Dijkstra, “A Note on Two Problems in Connexion with Graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.
2. P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009.
4. R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.
5. Python Software Foundation, Python 3 Documentation, standard-library documentation for heapq, collections, math, random and time.
6. Matplotlib Development Team, Matplotlib Documentation, plotting API used by the supplied implementation.